

Constructor Injection vs Setter vs Field Injection

What is Dependency Injection ?

Dependency Injection (DI) means:

In Spring Boot, objects do not build their dependencies — the framework (Creates and Supplies) injects them automatically at runtime.

Without DI:

```
class OrderService {  
    private PaymentService paymentService = new PaymentService();  
}
```

This is **tight coupling**.

With DI:

```
@Component  
class OrderService {  
    private final PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

Now Spring controls object creation.

Dependency Injection (DI) in Spring Boot is a design principle where the Spring container creates, manages, and supplies required objects (dependencies) to a class, instead of the class creating them itself.

This allows:

- Loose coupling
- Easier testing
- Better maintainability
- Scalable enterprise design

Dependency Inversion Principle (SOLID)

DIP says:

High-level modules should not depend on low-level modules.

Both should depend on abstractions.

Bad:

```
class OrderService {  
    private RazorPayPayment payment = new RazorPayPayment();  
}
```

Good:

```
class OrderService {  
    private Payment payment;  
}
```

Spring injects the implementation.

Types of Dependency Injection

Spring supports:

1. Field Injection
2. Setter Injection
3. Constructor Injection

Field Injection (First – Worst)

Code

```
@Component  
class OrderService {  
  
    @Autowired  
    private PaymentService paymentService;  
}
```

How it works internally

1. Spring creates object using default constructor
2. Uses reflection
3. Injects field later

Problems

Issue	Why
NPE risk	Object can be used before injection
Not immutable	Cannot mark field final
Hidden dependencies	Not visible in constructor
Hard to test	Must use Spring context
Breaks encapsulation	Reflection bypass

Setter Injection

Code

```
@Component
class OrderService {

    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

How it works

1. Object created
2. Setter called
3. Dependency injected later

Advantages

Benefit	Reason
Optional dependencies	Can skip setter
Easy mocking	Can replace dependency
Flexible	Can change dependency

Disadvantages

Issue	Why
Not immutable	Cannot mark final
Object incomplete	Dependency set later
Harder to read	Dependencies hidden

Constructor Injection (Enterprise Standard)

Code

```
@Component
class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Key Rules

Rule	Explanation
@Autowired optional	If only one constructor
Multiple dependencies	Allowed
Fields can be final	Immutability
Injected at creation	Object never incomplete

Why Constructor Injection is Preferred ?

Feature	Constructor Injection	Setter Injection	Field Injection
Immutability	✓	✗	✗
Fail fast	✓	✗	✗
Testability	✓	⚠	✗
Clear design	✓	⚠	✗
Thread-safe	✓	✗	✗

Constructor Injection enforces **correct object creation**.

Fail Fast Behavior

Missing dependency:

```
@Component
class OrderService {
    public OrderService(PaymentService paymentService) { }
}
```

If bean missing → **Application fails at startup** → This is good

But Field Injection fails at runtime with NPE → This is bad

Circular Dependency Behaviour

Field Injection allows cycle:

```
@Component
class A {
    @Autowired B b;
}

@Component
class B {
    @Autowired A a;
}
```

Spring can break using proxies.

Constructor Injection:

```
@Component
class A {
    public A(B b) { }
}

@Component
class B {
    public B(A a) { }
}
```

Fails immediately → **Correct design enforcement**

Unsatisfied Dependency

Unsatisfied Dependency occurs when Spring cannot find or create a bean that another bean depends on.

Spring knows *what you need*, but it cannot find *what to inject* → So application startup fails.

Because Spring does **Dependency Injection at startup**.

If even one dependency is missing → Spring stops the application.

Example

Step 1: Service depends on Repository

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;

    @Autowired
    public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }
}
```

Spring must find a bean of type OrderRepository.

Step 2: Repository is NOT a Spring Bean

```
public class OrderRepository {  
}
```

Now when Spring starts:

```
OrderService needs OrderRepository  
But OrderRepository is not a bean
```

Step 3: Application crashes

Typical error:

Unsatisfied dependency expressed through constructor parameter 0;

No qualifying bean of type 'OrderRepository' available

Meaning:

I tried to inject OrderRepository, but no such bean exists.

Correct Version

```
@Repository  
public class OrderRepository {  
}
```

Now Spring registers it as a bean.

Startup succeeds.

Multiple Beans Problem

```
interface Order {}

@Component
class OnlineOrder implements Order {}

@Component
class OfflineOrder implements Order {}
```

Injection:

```
@Autowired
private Order order;
```

✗ Error: 2 beans found.

Solution for Multiple beans problem

@Primary

```
@Component
@Primary
class OnlineOrder implements Order {}
```

Now Spring picks OnlineOrder by default.

@Qualifier

```
@Autowired
@Qualifier("offlineOrder")
private Order order;
```

Explicit selection.

@Primary vs @Qualifier Precedence

Used Together	Winner
Both present	@Qualifier

Using @Primary and @Qualifier in All Injections

Field

```
@Autowired
@Qualifier("onlineOrder")
private Order order;
```

Setter

```
@Autowired
public void setOrder(@Qualifier("onlineOrder") Order order) { }
```

Constructor

```
public OrderService(@Qualifier("onlineOrder") Order order) { }
```

Summary of Multiple Implementations Problem

If interface has 2 implementations → Spring confused.

Solution:

- Use @Primary for default
- Use @Qualifier for explicit control

Hidden Traps

Trap	Explanation
Field injection passes interviews	Fails in real systems
Circular deps hidden	Field injection masks design flaws
Proxy confusion	Lazy beans hide cycles
Unit testing pain	Field injection blocks mocking

One-Page Mental Model

Field → Convenient but dangerous

Setter → Flexible but incomplete objects

Constructor → Safe, strict, production-grade

Top Interview Questions

1. Why is constructor injection preferred?
→ Immutability + fail-fast + testability
2. When is @Autowired optional?
→ Single constructor
3. Can we autowire two constructors?
→ No
4. Which injection best for mandatory deps?
→ Constructor
5. Why field injection is bad?
→ Hidden deps, no immutability
6. How to resolve multiple beans?
→ @Primary / @Qualifier
7. Which has higher priority?
→ @Qualifier
8. Does constructor injection allow circular deps?
→ No (by design)
9. Why is that good?
→ Enforces better architecture
10. Which injection best for unit testing?
→ Constructor

Must-Know Dependency Injection Annotations in Spring Boot

@Autowired – Injects dependency automatically
@Qualifier – Chooses one bean when multiple exist
@Primary – Sets default bean
@Component – Makes a class a Spring bean
@Service – Business layer bean
@Repository – DAO layer bean
@Configuration – Configuration class
@Bean – Manually creates a bean
@Inject – Java standard alternative to @Autowired
@Resource – Injects by name first, then type

Final Summary — As a Story

Spring Initializr creates project.

pom.xml defines dependencies.

Spring Boot auto-configures beans.

When ApplicationContext starts:

Spring creates each bean.

With Field Injection → object created first, fixed later

With Setter Injection → object created, modified later

With Constructor Injection → object created correctly once

Enterprise systems demand **correct objects from birth** — not patched later.

That is why Constructor Injection dominates real systems.